

Programação Lógica

Recursão

Prof. Edson Alves

Campus UnB Gama: Faculdade de Ciências e Tecnologias em Engenharia

Recursão em Prolog

Recursão em Prolog

- ★ Em Prolog, predicados podem ser definidos recursivamente

Recursão em Prolog

- ★ Em Prolog, predicados podem ser definidos recursivamente
- ★ Um predicado é definido **recursivamente** se uma ou mais regras em sua definição se referem ao próprio predicado

Recursão em Prolog

- ★ Em Prolog, predicados podem ser definidos recursivamente
- ★ Um predicado é definido **recursivamente** se uma ou mais regras em sua definição se referem ao próprio predicado
- ★ Uma definição recursiva é composta de duas partes:
 1. uma ou mais cláusulas-base; e
 2. uma ou mais regras recursivas

Recursão em Prolog

- ★ Em Prolog, predicados podem ser definidos recursivamente
- ★ Um predicado é definido **recursivamente** se uma ou mais regras em sua definição se referem ao próprio predicado
- ★ Uma definição recursiva é composta de duas partes:
 1. uma ou mais cláusulas-base; e
 2. uma ou mais regras recursivas
- ★ Cláusulas-base devem ser fatos ou generalizações

Recursão em Prolog

- ★ Em Prolog, predicados podem ser definidos recursivamente
- ★ Um predicado é definido **recursivamente** se uma ou mais regras em sua definição se referem ao próprio predicado
- ★ Uma definição recursiva é composta de duas partes:
 1. uma ou mais cláusulas-base; e
 2. uma ou mais regras recursivas
- ★ Cláusulas-base devem ser fatos ou generalizações
- ★ O tratamento das regras recursivas se dá naturalmente pelo *backtracking*

% Implementação recursiva da função fatorial

`fact(N, F) :-`

`N > 0,`

`succ(NewN, N),`

`fact(NewN, F1),`

`F is F1*N.`

`fact(0, 1).`

% Implementação recursiva de cauda

`factTR(N, Acc, F) :-`

`N > 0,`

`succ(NewN, N),`

`NewAcc is Acc * N,`

`factTR(NewN, NewAcc, F).`

`factTR(0, Acc, Acc).`

`factorial(N, F) :- factTR(N, 1, F).`

Características da recursão em Prolog

Características da recursão em Prolog

- ★ Cada nível de recursão tem seu próprio conjunto de variáveis

Características da recursão em Prolog

- ★ Cada nível de recursão tem seu próprio conjunto de variáveis
- ★ A unificação entre o objetivo e o padrão de uma regra recursiva definem as relações entre as variáveis de diferentes níveis

Características da recursão em Prolog

- ★ Cada nível de recursão tem seu próprio conjunto de variáveis
- ★ A unificação entre o objetivo e o padrão de uma regra recursiva definem as relações entre as variáveis de diferentes níveis
- ★ Em uma regra recursiva, é preciso garantir que os parâmetros não caracterizem uma dentre as cláusulas-base

Características da recursão em Prolog

- ★ Cada nível de recursão tem seu próprio conjunto de variáveis
- ★ A unificação entre o objetivo e o padrão de uma regra recursiva definem as relações entre as variáveis de diferentes níveis
- ★ Em uma regra recursiva, é preciso garantir que os parâmetros não caracterizem uma dentre as cláusulas-base
- ★ Sem esse cuidado, um retorno pela porta **redo** em alguma das subconsultas da regra pode fazer com que, eventualmente, um conjunto de valores que caracteriza uma cláusula-base avance na regra recursiva, causando erros de execução como laços infinitos

% Computa o n-ésimo número de Fibonacci

fibTR(N, B, A, F) :-

 N > 0,

 succ(NewN, N),

 C is A + B,

 fibTR(NewN, C, B, F).

fibTR(0, _, Acc, Acc).

:- use_module(library(arithmetic)).

:- arithmetic_function(fib/1).

fib(N, F) :- fibTR(N, 1, 0, F).

% Exemplo de consulta. SWI-Prolog implementa aritmética estendida

?- X is fib(200).

X = 280571172992510140037611932413038677189525.

Implementação iterativa

```
def fib(N):  
    if N < 2:  
        return N
```

```
    a = 0
```

```
    b = 1
```

```
    for _ in range(N):  
        a, b = b, a + b
```

```
    return a
```

Computa o bilionésimo número de Fibonacci em Python

Tempo de execução: 9,251s

```
print(fib(10**6))
```

% Computa o n-ésimo número de Fibonacci

fibTR(N, B, A, F) :-

 N > 0,

succ(NewN, N),

 C is A + B,

fibTR(NewN, C, B, F).

fibTR(0, _, Acc, Acc).

:- use_module(**library**(**arithmetic**)).

:- arithmetic_function(**fib**/1).

fib(N, F) :- **fibTR**(N, 1, 0, F).

% Computa o bilionésimo número de Fibonacci em ProLog

% Tempo de execução: 6,491s

?- profile(X is **fib**(1000000)).

Estruturas de dados em Prolog

Estruturas de dados em Prolog

- ★ Uma estrutura de dados combina termos primitivos (átomos, inteiros, etc) e estruturas em tipos compostos

Estruturas de dados em Prolog

- ★ Uma estrutura de dados combina termos primitivos (átomos, inteiros, etc) e estruturas em tipos compostos
- ★ A sintaxe de declaração de uma estrutura de dados é

`functor(arg1, arg2, ..., argN).`

Estruturas de dados em Prolog

- ★ Uma estrutura de dados combina termos primitivos (átomos, inteiros, etc) e estruturas em tipos compostos

- ★ A sintaxe de declaração de uma estrutura de dados é

`functor(arg1, arg2, ..., argN).`

- ★ Cada argumento pode ser um tipo primitivo ou outra estrutura

Estruturas de dados em Prolog

- ★ Uma estrutura de dados combina termos primitivos (átomos, inteiros, etc) e estruturas em tipos compostos

- ★ A sintaxe de declaração de uma estrutura de dados é

```
functor(arg1, arg2, ..., argN).
```

- ★ Cada argumento pode ser um tipo primitivo ou outra estrutura

- ★ A declaração de uma estrutura é semelhante à declaração de um fato

```
car(peugeot, black, 2).  
car(honda, red, 4).
```

Consultas envolvendo estruturas de dados

Consultas envolvendo estruturas de dados

- ★ A ordem dos argumentos influencia o resultado de um consulta

```
car(X, red, _).
```

Consultas envolvendo estruturas de dados

- ★ A ordem dos argumentos influencia o resultado de um consulta

```
car(X, red, _).
```

- ★ Campos podem ser ignorados com a variável anônima ‘_’

Consultas envolvendo estruturas de dados

- ★ A ordem dos argumentos influencia o resultado de um consulta

```
car(X, red, _).
```

- ★ Campos podem ser ignorados com a variável anônima ‘_’
- ★ Estruturas podem ser aninhadas, com o intuito de aumentar a legibilidade

```
car(honda, color(red), doors(4)).
```

Consultas envolvendo estruturas de dados

- ★ A ordem dos argumentos influencia o resultado de um consulta

```
car(X, red, _).
```

- ★ Campos podem ser ignorados com a variável anônima ‘_’
- ★ Estruturas podem ser aninhadas, com o intuito de aumentar a legibilidade

```
car(honda, color(red), doors(4)).
```

- ★ As duas cláusulas acima não unificam

% Cada matrioska pode conter outras de tamanho menor

matrioska(a, b).

matrioska(a, c).

matrioska(b, d).

matrioska(b, c).

matrioska(b, f).

matrioska(c, e).

matrioska(c, g).

matrioska(d, g).

matrioska(e, f).

matrioska(f, g).

matrioska(f, h).

% Função recursiva auxiliar

`list_(M, L) :-`

`nl,`

`tab(L),`

`print(->),`

`tab(1),`

`print(M),`

`matrioska(M, X),`

`NewL is L + 4,`

`list_(X, NewL).`

% Lista todas as matrioskas contidas em M a partir da coluna L

`list(M, L) :- list_(M, L).`

`list(_, _).`

Referências

- ★ **BLACKBURN**, Patrick, **BOS**, Johan, **STRIEGNITZ**, Kristina. *Learn Prolog Now!*, Online, 2026.
- ★ **MERRIT**, Dennis. *Adventure in Prolog*, Amzi!, 191 pgs, 2017.
- ★ **SWI-Prolog**. <https://www.swi-prolog.org/>, acesso em 13/03/2026.
- ★ **Wikipédia**. *Prolog*, acesso em 13/03/2026.